

INFORME TECNICO GORILLA.BAS



CARRERA: Ingeniería Informática

MATERIA: PROGRAMACION I

ALUMNO: Santtino Gómez

PROFESOR: Gustavo Quiñonez

Investigación del código generado con IA

1. Estructura General del Programa

- **¿Dónde comienza la ejecución?:** La ejecución inicia en `int main()`. Ahora este bloque contiene un bucle `do-while` que no solo muestra el menú, sino que permite reiniciar el juego por completo sin cerrar la consola.
- **¿Cuál es la lógica general?:** El programa utiliza una estructura de menús anidados: primero elige el modo (Jugador vs. Jugador o Jugador vs. CPU) y luego el tipo de partida (Ronda única o Mejor de 3).
- **¿Cómo se organiza el flujo?:** Se delega la lógica a la función `jugar()`, la cual ahora gestiona un sistema de victorias acumuladas. El flujo solo regresa al menú principal cuando un jugador alcanza las victorias necesarias (1 o 2).

2. Funciones del Programa

1. main

- **Qué hace:** Es el punto de inicio del programa. Controla el flujo principal, gestiona el menú de selección de modo (vs. CPU o vs. J2) y permite reiniciar el juego al finalizar una partida.
- **Parámetros:** Ninguno.
- **Qué devuelve:** Un entero (int) que indica el estado de finalización del programa.
- **En qué parte del juego se utiliza:** Se ejecuta durante toda la vida del proceso, actuando como el contenedor principal.

2. jugar

- **Qué hace:** Gestiona el ciclo de vida de una partida completa. Controla el sistema de victorias para el "Mejor de 3", alterna los turnos entre jugadores y decide si la entrada de datos proviene del teclado o de la lógica de la CPU.
- **Parámetros:** bool contraCPU (determina si el segundo jugador es la máquina) y bool mejorDeTres (determina cuántas victorias se necesitan para ganar).
- **Qué devuelve:** No devuelve nada (void).
- **En qué parte del juego se utiliza:** Se invoca desde el main cuando el usuario selecciona iniciar una partida.

3. inicializarMapa

- **Qué hace:** Configura el escenario de juego. Limpia la matriz, genera 8 edificios con alturas aleatorias y posiciona a los jugadores en los edificios de los extremos.
- **Parámetros:** La matriz del mapa y las referencias a las estructuras de los dos jugadores (Jugador &j1, Jugador &j2).
- **Qué devuelve:** No devuelve nada (void).
- **En qué parte del juego se utiliza:** Se llama al inicio de cada nueva ronda para crear un escenario fresco.

4. disparar

- **Qué hace:** Ejecuta la simulación física del proyectil. Calcula la trayectoria punto por punto usando la gravedad modificada a 10 y detecta si hubo colisión con un edificio, el suelo o el oponente.
- **Parámetros:** El mapa, el tirador, el objetivo, el ángulo, la velocidad y un booleano para la dirección del disparo.
- **Qué devuelve:** Un valor booleano (true si hubo impacto en el rival, false en caso contrario).
- **En qué parte del juego se utiliza:** Se activa cada vez que un jugador (humano o CPU) confirma su disparo.

5. dibujarMapa

- **Qué hace:** Renderiza visualmente la matriz en la consola. Además, si el modo "Mejor de 3" está activo, muestra el marcador con las victorias acumuladas de cada jugador.
- **Parámetros:** La matriz del mapa, los contadores de victorias de ambos jugadores y la bandera para mostrar o no el marcador.
- **Qué devuelve:** No devuelve nada (void).
- **En qué parte del juego se utiliza:** Se invoca repetidamente para actualizar la imagen que ve el usuario, especialmente antes y después de cada disparo.

6. leerEntradaValida

- **Qué hace:** Funciona como un filtro de seguridad para la entrada de datos. Solicita un número al usuario y, si este ingresa texto o caracteres no válidos, limpia el error y vuelve a preguntar para evitar que el programa se bloquee.
- **Parámetros:** Una cadena de texto (string mensaje) que se muestra al usuario como instrucción.
- **Qué devuelve:** El valor numérico validado (double).
- **En qué parte del juego se utiliza:** Se usa cada vez que se requiere una elección en el menú o el ingreso de potencia y ángulo.

3. Identificación de librerías

- `<iostream>`: Para entrada/salida de datos (cin, cout).
- `<cmath>`: Para las funciones sin(), cos() y el valor de PI.
- `<cstdlib>`: Para usar rand() (edificios y CPU) y system("cls") (limpiar pantalla).
- `<ctime>`: Para que srand use el reloj del sistema y los edificios siempre sean diferentes.
- `<string>`: Para manejar los nombres de los jugadores y el objeto string.

4. Variables y Estructuras Importantes

- **const double GRAVEDAD = 10.0**: Nueva constante que acelera la caída de los proyectiles.
- **struct Jugador**: Ahora incluye la variable bool esCPU, necesaria para decidir si el programa debe pedir datos al teclado o autogenerarlos.
- **int victoriasJ1 / victoriasJ2**: Acumuladores esenciales para el modo "Mejor de 3".
- **int tirosJ1 / tirosJ2**: Contadores que permiten mostrar el resumen final de cuántos disparos realizó cada uno.
- **char mapa[ALTO][ANCHO]**: Matriz de 25x110. Se aumentó el ancho para dar espacio a los 8 edificios.

5. Lógica del Disparo

- **Cálculo de Trayectoria**: Se basa en el tiempo incremental ($t += 0.1$). La posición horizontal se suma para J1 y se resta para J2 (ya que dispara hacia la izquierda).
- **Detección de Impacto**: El sistema verifica si la coordenada (xActual, yActual) coincide con el símbolo del oponente. Si el proyectil toca un #, se detiene y marca una X en el edificio.
- **Mejora de Visualización**: El programa imprime la trayectoria del último tiro para que los jugadores puedan corregir su puntería.

6) Uso de IA

- ¿Qué prompts utilizó?

Quiero desarrollar un juego simple en el lenguaje C++ de dos jugadores basado en disparos de proyectiles, tipo “Gorillas.bas” con una representación visual sencilla del escenario.

El programa debe cumplir con los siguientes requisitos mínimos:

Características del juego:

- Participan dos jugadores que se turnan.
- Existe un escenario con edificios o alguna representación equivalente.
- Cada jugador debe ingresar un ángulo y una velocidad para realizar su disparo.
- El sistema debe calcular la trayectoria del proyectil.
- Se debe detectar si el proyectil impacta al oponente o no.
- El juego debe alternar turnos automáticamente.
- Se debe indicar claramente el ganador de cada ronda o de la partida.
- Debe existir la opción de reiniciar o volver a jugar.
- El escenario debe tener una representación visual básica (puede ser gráfica o en consola).

Requisitos técnicos del código:

El programa debe incluir:

- Funciones bien definidas y separadas.
- Uso correcto de variables y tipos de datos.
- Validación y control de entrada de datos.
- Uso de estructuras condicionales (if, else).
- Uso de estructuras repetitivas (loops).
- Modularización del código.
- Comentarios técnicos claros en las partes importantes.

Además, necesito que el programa esté bien documentado respondiendo lo siguiente:

1. Estructura general del programa:

- ¿Dónde comienza la ejecución?
- ¿Cuál es la lógica general del programa?
- ¿Cómo se organiza el flujo del juego?

2. Funciones del programa:

Identificar al menos 6 funciones indicando:

- Nombre de la función
- Qué hace
- Qué parámetros recibe
- Qué devuelve (si aplica)
- En qué parte del juego se utiliza

3. Librerías utilizadas:

Explicar qué librerías se usan y para qué sirven, por ejemplo:

- Entrada/salida
- Cálculos matemáticos
- Números aleatorios
- Manejo de cadenas
- Gráficos (si se utilizan)

4. Variables y estructuras importantes:

Explicar:

- Qué variables, arreglos o estructuras se utilizan
- Qué almacenan
- Por qué son necesarias
- Cuándo se actualizan durante el juego

5. Lógica del disparo:

Explicar de forma simple:

- Cómo se calcula la trayectoria del proyectil
- Qué variables intervienen
- Cómo se detecta si hubo impacto o fallo

El resultado debe ser un programa funcional, claro, bien estructurado y fácil de entender, ideal para fines educativos.

- ¿Qué partes del código fueron generadas por IA?

La mayor parte del código fue hecha por IA, ya que se usó como base para poder mejorar y modificar el código con el fin de una mejor personalización y modificación de este.

- ¿Qué partes corrigió o ajustó manualmente?

Las partes corregidas y ajustadas fueron la física del disparo del proyectil, la forma del escenario, se agregaron nuevos modos de juego, se proporcionó más información al término de cada partida.

- ¿Qué errores encontró en la respuesta de la IA?

Se encontraron errores de proporcionalidad y manejo visual de los edificios al momento de la ejecución de la batalla, solo había dos edificios (Cada jugador situándose arriba de su edificio propio) en vez de varios.

Modificaciones manuales

1. Modificación de la física del disparo

- **Qué línea o bloque de código se modificó:** En la sección de constantes al inicio del código, se cambió `const double GRAVEDAD = 9.8;` por `const double GRAVEDAD = 10.0;`. Esta constante se utiliza dentro de la función `disparar()` en la ecuación de movimiento parabólico: $(0.5 * GRAVEDAD * t * t)$.
- **Qué efecto produjo el cambio:** El proyectil ahora cae ligeramente más rápido. La curva de la parábola se vuelve un poco más "cerrada" o pronunciada hacia abajo en comparación con la gravedad anterior.
- **Por qué ese cambio altera la trayectoria:** Al aumentar la gravedad, la fuerza de atracción hacia el suelo (el eje Y de nuestra matriz) es mayor por cada décima de segundo (\$t\$) simulada. Físicamente, esto reduce el tiempo de vuelo y la altura máxima alcanzada por el proyectil a menos que se compense ingresando una mayor velocidad inicial.

2. Modificación del escenario

- **Cómo funcionaba antes:** El código generaba solamente 2 edificios; uno a la izquierda utilizando un bloque `for` de ancho 10 (de la columna 2 a la 12), y otro a la derecha simétrico. Los jugadores se colocaban en las coordenadas fijas de esos dos únicos edificios.
- **Cómo funciona después del cambio:** Se agregaron las constantes `NUM_EDIFICIOS = 8` y `ANCHO_EDIFICIO = 12` (dos columnas más que el anterior). En la función `inicializarMapa`, ahora existe un bucle `for(int b = 0; b < NUM_EDIFICIOS; b++)` que dibuja 8 edificios lado a lado con un espacio de separación. El Jugador 1 se coloca dinámicamente encima del edificio en el índice 0, y el Jugador 2 en el edificio en el índice 7 (el octavo).
- **Qué efecto visual o lógico se produjo:** Visualmente, la pantalla pasó de verse vacía en el centro a tener una densa "ciudad" con distintas alturas que actúan como múltiples obstáculos. Lógicamente, hace el juego mucho más difícil porque cualquier disparo con un ángulo muy bajo chocará con los edificios intermedios antes de llegar al oponente.

3. Modificación de una regla del juego (Al mejor de 3 y Reinicio)

- **Qué función o bloque controla esa lógica:**
 - **Reinicio:** Se controla en la función main() usando un bucle do-while global.
 - **Mejor de 3:** Se controla en la función jugar(), usando las variables victoriasJ1, victoriasJ2 y victoriasNecesarias dentro del bucle while (victoriasJ1 < victoriasNecesarias && victoriasJ2 < victoriasNecesarias).
- **Qué se agregó o modificó:** En main(), luego de elegir si jugar contra CPU o contra un amigo, se agregó un segundo menú preguntando si se quiere "Ronda única" o "Mejor de 3". Se modificó la firma de la función para pasar el parámetro bool mejorDeTres. Además, se agregó el parámetro bool mostrarMarcador a dibujarMapa() para que el texto de puntaje solo se imprima si se seleccionó ese modo.
- **Cómo se verificó que el cambio funciona:** Al compilar, si eliges "Mejor de 3", acertar un impacto no termina el programa, sino que suma 1 al marcador, limpia el escenario y arranca la Ronda 2. El programa finaliza solo cuando un contador llega a 2, mostrando la opción de volver al menú de inicio.

4. Modificación de la interfaz o salida de información

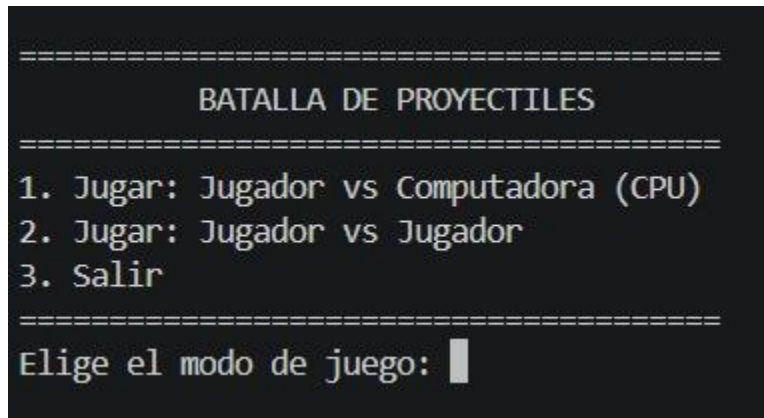
- **Qué parte del código controla la salida de información:** Dentro de la función jugar(), en el bucle interior que controla los turnos, y al final de esa misma función.
- **Cómo se modificó:**
 1. Antes de pedir los datos, se agregó un cout indicando claramente el turno y la ronda (cout << "Turno actual: " << actual.nombre).
 2. Al confirmar los datos (o si la CPU los genera), se imprimen en pantalla.
 3. Se crearon las variables tirosJ1 y tirosJ2 que incrementan en 1 cada vez que se dispara. Al romper el bucle por alcanzar las victorias, se limpia la pantalla y se imprime un bloque de "Estadísticas Finales".
- **Qué mejora aporta al usuario:** Aporta "Feedback" (retroalimentación). El usuario ahora sabe exactamente en qué estado está el juego. El resumen final añade competitividad al revelar la "eficacia" (quién necesitó menos disparos para ganar).

5. Mejora libre obligatoria: Jugador vs Computadora (CPU)

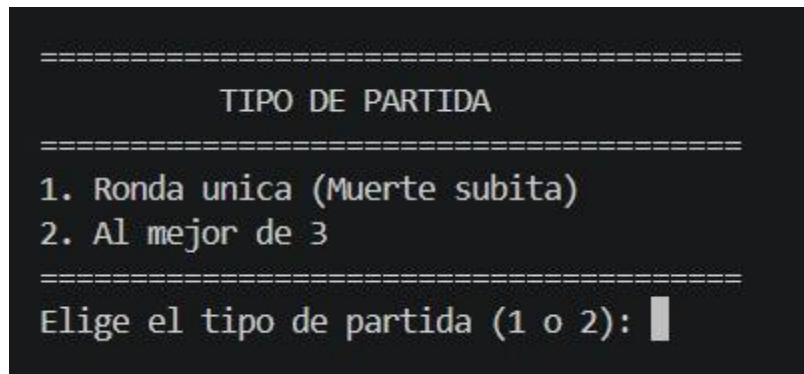
- **Qué se agregó:** Un modo de juego "PvE" (Player vs Environment/Computer).
- **Cómo funciona:** Si el usuario selecciona jugar contra la CPU en el menú principal, la bandera `contraCPU` se vuelve `true`. Esto hace que el Jugador `j2` se inicialice con la propiedad `esCPU = true`. Cuando es el turno de `J2`, el programa se salta la lectura de teclado (`cin`) y usa `rand()` para auto-generar un ángulo entre 20 y 70, y una velocidad entre 10 y 35, imprimiendo su decisión en consola para que el jugador sepa qué hizo la máquina.
- **En qué parte del código se implementó:** Se modificó la estructura Jugador para incluir `bool esCPU`. Se actualizó el `main()` y `mostrarMenuPrincipal()` para dar la opción, y en `jugar()`, se añadió un bloque `if (actual.esCPU)` que automatiza la introducción de datos justo antes de invocar a la función `disparar()`.
- **Qué valor aporta al juego:** Permite que el juego sea disfrutable por una sola persona. Antes, para probar o practicar el juego, el usuario debía ingresar los datos de ambos bandos, lo cual era tedioso. Ahora, la máquina ofrece una respuesta autónoma.

Anexo

- Capturas de pantalla del funcionamiento del programa



Menú principal.



Menú de selección del tipo de partida (Se despliega luego de seleccionar la opción 1, jugar contra la CPU o 2, jugar contra una persona real).

